

컴퓨터비전_인공지능(AI)개론

16. CNN 스크래치 구현 Residual block_ResNet18

- CNN 구조 이해하기
- 아키텍처 설계
- 실전 구현

▶ 기존 MLP 근본적 한계

문제 1: 공간 정보 손실

이미지를 1D 벡터로 변환하는 과정에서 픽셀 간 공간적 관계가 완전히 소실됩니다. 32x32 RGB 이미지는 3,072개의 독립적인 입력 뉴런으로 처리되어, 인접 픽셀의 상관 관계를 전혀 활용하지 못합니다.

문제 2: 파라미터 폭발

256x256 이미지를 1,000개의 hidden unit 과 연결하면 196M개의 파라미터가 필요합니다. 이는 과적합 위험을 극도로 높이고 학습을 사실상 불가능하게 만듭니다.

문제 3: 위치 불변성 부족

동일한 객체가 이미지 내 다른 위치에 있으면 완전히 새로운 특징으로 인식됩니다. 평행이동, 회전 등의 변환에 매우 취약한 구조입니다.

- Local Connectivity : 지역적 패턴 인식
- Parameter Sharing : 필터 재사용
- Spatial Hierarchy : 계층적 특징 학습

합성곱의 핵심 개념

합성곱은 입력 이미지와 필터(커널)의 element-wise 곱셈을 수행한 후 합산하는 연산입니다. 슬라이딩 윈도우 방식으로 전체 이미지를 순회하며, 각 위치에서 특정 패턴의 존재 여부를 검출합니다.

수학적 정의

$$Output(i, j) = \sum_m \sum_n Input(i + m, j + n) \times Kernel(m, n)$$

이 연산을 통해 에지, 코너, 텍스처와 같은 저수준 특징을 자동으로 검출할 수 있으며, 필터의 가중치는 역전파 학습을 통해 최적화됩니다.

합성곱 연산의 가장 큰 장점은 **translation invariance**입니다. 동일한 필터가 이미지 전체를 순회하므로, 특정 패턴이 어느 위치에 있든 일관되게 검출할 수 있습니다.

Edge Detection 예제

3×3 Sobel 필터:

```
[ -1  -1  -1 ]
[  0   0   0 ]
[  1   1   1 ]
```

수평 에지를 검출하여 경계선을 강조합니다. 필터의 값이 클수록 강한 활성화를 나타냅니다.

필터 (Filter/Kernel)

일반적으로 3x3, 5x5, 7x7 크기를 사용합니다. 필터의 깊이는 입력 채널 수와 동일하며, 필터 개수는 출력 Feature Map의 채널 수를 결정합니다. VGG, ResNet 등 현대적 아키텍처는 주로 3x3 필터를 사용합니다.

스트라이드 (Stride)

필터가 이동하는 간격을 제어합니다. stride=1은 한 칸씩, stride=2는 두 칸씩 이동하여 출력 크기를 1/2로 축소합니다. 큰 stride는 계산량을 줄이지만 정보 손실이 발생할 수 있습니다.

패딩 (Padding)

입력 이미지 주변에 0을 추가하여 출력 크기를 제어합니다. Valid Padding은 패딩 없이 크기가 감소하고, Same Padding은 출력 크기를 입력과 동일하게 유지합니다.

출력 크기 계산 공식

$$Output = \frac{Input - Kernel + 2 \times Padding}{Stride} + 1$$

실전 예제:

- Input: 32x32, Kernel: 5x5
- Stride: 1, Padding: 0
- Output: $(32-5)/1+1 = 28 \times 28$

PyTorch 구현

```
conv = nn.Conv2d(
    in_channels=3,
    out_channels=64,
    kernel_size=5,
    stride=1,
    padding=2
)
# 출력: 32x32x64
```

합성곱 연산의 출력인 Feature Map은 입력 이미지에서 특정 패턴이 어디에 존재하는지를 나타냅니다. 각 필터는 하나의 Feature Map을 생성하며, 다중 필터를 사용하면 다양한 특징을 동시에 검출할 수 있습니다.

01

입력 이미지

32×32×3 (RGB)

02

첫 번째 합성곱

32 filters → 32×32×32

03

두 번째 합성곱

64 filters → 32×32×64

04

특징 추출 완료

64개 Feature Maps

ReLU 활성화 함수

$$\text{ReLU}(x) = \max(0, x)$$

ReLU는 CNN에서 가장 널리 사용되는 활성화 함수입니다. 계산이 매우 빠르고(단순 max 연산), Gradient Vanishing 문제를 완화하며, 희소성(Sparsity)을 제공하여 효율적인 표현을 가능하게 합니다.

다른 활성화 함수

- **LeakyReLU**: 음수 영역에서 작은 기울기 유지
- **ELU**: 지수함수 기반 부드러운 곡선
- **GELU**: Transformer에서 주로 사용

차원 축소

공간 해상도를 줄여 계산량을 크게 감소시킵니다. 2x2 풀링은 크기를 1/4로 줄입니다.

위치 불변성

미세한 위치 변화에 강건한 표현을 학습합니다. 객체의 정확한 위치보다 존재 여부가 중요합니다.

수용 영역 확대

더 넓은 영역의 정보를 통합하여 고수준 특징을 형성합니다.

Max Pooling

윈도우 내에서 최대값을 선택하여 가장 두드러진 특징을 보존합니다. 예 지나 코너 같은 강한 활성화를 유지하는 데 효과적입니다.

Input (4×4):

```
[1 3 2 4]
[5 6 7 8]
[2 3 1 2]
[9 7 8 11]
```

Output (2×2):

```
[6 8]
[9 11]
```

Average Pooling

윈도우 내 평균값을 계산하여 전체적인 특징을 보존합니다. Global Average Pooling은 분류기 직전에 사용되어 각 Feature Map을 하나의 값으로 압축합니다.

PyTorch 구현

```
# Max Pooling
pool = nn.MaxPool2d(
    kernel_size=2,
    stride=2
)

# Global Average Pooling
gap = nn.AdaptiveAvgPool2d((1, 1))
```

❏ **중요:** 풀링 레이어는 학습 가능한 파라미터가 없습니다. 너무 많은 풀링은 정보 손실을 유발하므로 적절한 균형이 필요합니다.



LeNet-5 구조 예시

1. Input: 32x32x1 (Grayscale)
2. Conv1: 6 filters (5x5) → 28x28x6
3. Pool1: Max (2x2) → 14x14x6
4. Conv2: 16 filters (5x5) → 10x10x16
5. Pool2: Max (2x2) → 5x5x16
6. Flatten → 400 features
7. FC1: 400 → 120
8. FC2: 120 → 84
9. FC3: 84 → 10 classes

설계 핵심 원칙

- **채널 수 증가**: 깊이가 깊어질수록 32 → 64 → 128 → 256으로 증가
- **공간 크기 감소**: 풀링을 통해 점진적으로 축소
- **계층적 학습**: Low-level → High-level 특징

▶ CNN 두 가지 핵심 구성 요소



▶ Pytorch 구현 예시

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Feature Extractor
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )

        # Classifier
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*8*8, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        x = self.features(x)    # 특징 추출
        x = self.classifier(x)  # 분류
        return x
```



필터 개수 (Number of Filters)

적을수록 빠르지만 표현력이 감소하고, 많을수록 표현력이 증가하지만 과적합 위험이 커집니다.

일반적 패턴: 32 → 64 → 128 → 256 (2배씩 증가)

커널 크기 (Kernel Size)

3×3: VGG, ResNet에서 사용하는 가장 일반적인 크기

5×5, 7×7: AlexNet 등 초기 레이어에서 사용

1×1: 채널 수 조정, Bottleneck 구조에 활용

레이어 깊이 (Depth)

얕은 네트워크는 간단한 특징만 학습하지만, 깊은 네트워크는 복잡한 특징을 학습할 수 있습니다. 단, 너무 깊으면 Gradient Vanishing 문제가 발생합니다.

트레이드오프 분석

요소	작은 값	큰 값
커널 크기	파라미터 ↓, 깊은 네트워크	넓은 수용 영역, 파라미터 ↑
필터 개수	빠른 학습, 표현력 ↓	높은 표현력, 과적합 위험
깊이	간단한 특징	복잡한 특징, Vanishing

정규화 기법

- **Dropout:** FC layer에 $p=0.5$ 적용
- **Batch Normalization:** Conv layer 후 적용
- **Data Augmentation:** 학습 데이터 증강

📄 실험 가이드라인

Baseline: 2-3 Conv blocks, 32→64→128 filters, 3×3 kernels

개선: 레이어 추가, 필터 수 증가, 정규화 적용

- 데이터 셋 개요
 - 60,000 이미지 (32 * 32 컬러 이미지)
 - 훈련용 데이터 50K, 테스트 데이터 10K
 - 10개 클래스(각 6,000장)
- 데이터 특징
 - 저해상도 (32×32): 빠른 실험 가능
 - 실제 사진: 라벨 노이즈 존재
 - 높은 클래스 유사성: cat vs dog, automobile vs truck

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
]) # 픽셀 값 [0, 1] → [-1, 1]
```

▶ Simple CNN 아키텍처

설계 포인트

- 채널 수 증가: $3 \rightarrow 32 \rightarrow 64$ 로 점진적 확장
- 공간 축소: $32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$
- Feature 압축: $4096 \rightarrow 256$ 으로 차원 축소
- 출력 레이어: 10개 클래스에 대한 로짓

❏ FC 레이어의 파라미터가 전체의 대부분을 차지합니다. 이는 CNN에서 흔한 현상입니다.

```

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()

        # Feature Extractor
        self.features = nn.Sequential(
            # Conv Block 1
            nn.Conv2d(3, 32, kernel_size=3, padding=1), # 32×32×3 → 32×32×32
            nn.ReLU(),
            nn.MaxPool2d(2, 2), # 32×32×32 → 16×16×32

            # Conv Block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1), # 16×16×32 → 16×16×64
            nn.ReLU(),
            nn.MaxPool2d(2, 2) # 16×16×64 → 8×8×64
        )

        # Classifier
        self.classifier = nn.Sequential(
            nn.Flatten(), # 8×8×64 → 4096
            nn.Linear(64*8*8, 256), # 4096 → 256
            nn.ReLU(),
            nn.Linear(256, 10) # 256 → 10 classes
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

```

파라미터 계산

레이어	계산식	파라미터 수
Conv1	$3 \times 32 \times 3 \times 3 + 32$	896
Conv2	$32 \times 64 \times 3 \times 3 + 64$	18,496
FC1	$4096 \times 256 + 256$	1,048,832
FC2	$256 \times 10 + 10$	2,570
Total		~1.07M


```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for inputs, labels in loader:
        inputs, labels = inputs.to(device), labels.to(device)

        # Forward pass
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass
        loss.backward()
        optimizer.step()

        # Statistics
        running_loss += loss.item() * inputs.size(0)
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    epoch_loss = running_loss / total
    epoch_acc = 100. * correct / total
    return epoch_loss, epoch_acc
```

주요 구성 요소

- 손실 함수

CrossEntropyLoss: 다중 클래스 분류에 적합. Softmax와 NLL Loss를 결합한 형태입니다.

- 옵티마이저

Adam (lr=0.001): Adaptive learning rate를 사용하여 빠른 수렴을 보장합니다.

- 배치 크기

64: GPU 메모리와 학습 속도의 균형점입니다.

- 에폭 수

10-20: CIFAR-10에서 충분한 학습을 위한 권장 에폭입니다.

평가 함수 구현

```
def evaluate(model, loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

    return 100. * correct / total
```

❏ **model.eval()**: Dropout과 Batch Normalization을 평가 모드로 전환합니다.

torch.no_grad(): Gradient 계산을 비활성화하여 메모리를 절약합니다.

시각화 유형



학습 곡선

에폭에 따른 Loss와 Accuracy 변화를 시각화하여 학습 진행 상황과 과적합 여부를 판단합니다. 학습/검증 곡선의 간격이 벌어지면 과적합을 의심합니다.



Confusion Matrix

클래스별 혼동 행렬로 어떤 클래스 쌍이 자주 혼동되는지 확인합니다. cat-dog, automobile-truck처럼 시각적으로 유사한 클래스 간 오분류가 많습니다.



예측 결과

실제 이미지와 함께 정답 레이블 및 모델의 예측 결과를 표시합니다. 오분류된 샘플을 분석하여 모델의 약점을 파악할 수 있습니다.



Feature Map

각 합성곱 레이어의 활성화를 시각화하여 네트워크가 학습한 특징을 이해합니다. 초기 레이어는 에지, 후기 레이어는 고수준 패턴을 포착합니다.